

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Network Applications and Network
Administration
Filtering DNS Resolver

Contents

1	Introduction	2
2	Theoretical Background	2
2.1	What is Domain Name System (DNS)	2
2.2	DNS Message Structure	2
2.3	DNS Query Types	3
2.4	DNS Response Codes	3
3	Application Design	4
3.1	Code Structure	4
4	Implementation	4
4.1	Program Initialization	4
4.2	Query Handling	5
5	User Manual	5
5.1	Compilation	5
5.2	Usage	5
5.3	Example	6
6	Testing	6
6.1	Test Environment	6
6.2	Unit Tests	6
6.2.1	List of All Unit Tests	6
6.3	Manual Testing	7
6.3.1	Testing Working Domain	7
6.3.2	Testing Blocked Domain	7
6.4	Debug Output	7
6.5	Memory Leak Testing	7
7	Limitations	8
8	Conclusion	8

1 Introduction

In this project I implemented a filtering DNS resolver that acts as a middleman between clients DNS queries and an upstream DNS server. The purpose is to block DNS queries for the domains that are provided in a file and to forward others.

2 Theoretical Background

2.1 What is Domain Name System (DNS)

The domain name system is a hierarchical and distributed name service that provides a naming system for computers. It translates easily remembered domain names like `dns.google.com` to their IP addresses `8.8.4.4` that are needed to identify computer services and devices [1].

2.2 DNS Message Structure

The top level format of DNS message is divided into 5 sections. In some cases some of them are empty [2].

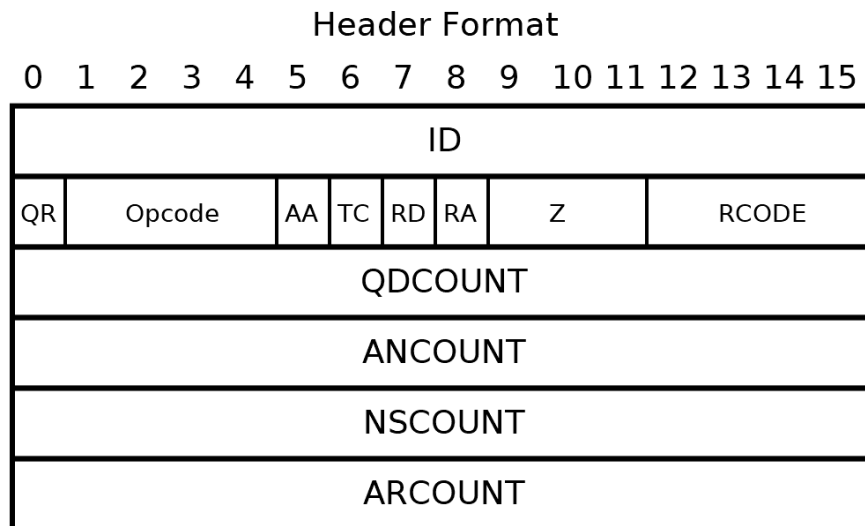


Figure 1: DNS Message Structure[3]

Header section includes fields that specify which of the remaining sections are present and also specifies whether the message is a query or a response, message identifier, opcode that specifies whether message is a standard query or other kind and also response code.

Question section is used to carry the question itself. It contains domain name, type of the query and class.

The answer, authority, and additional sections all share the same format: a variable number of resource records, where the number of records is specified in the corresponding count field in the header [2].

2.3 DNS Query Types

DNS supports various record types for different purposes, here are some of them [2]:

- **A (type 1)** – domain name to IPv4 address
- **NS (type 2)** – authoritative name server
- **SOA (type 6)** – marks the start of a zone of authority
- **MX (type 15)** – mail exchange server
- **PTR (type 12)** – domain name pointer
- **TXT (type 16)** – text string
- **AAAA (type 28)** – domain name to IPv6 address

2.4 DNS Response Codes

DNS Header includes an RCODE field that specifies the outcome of the query [2]:

- **0** – No error condition
- **1** – Format error - The name server was unable to interpret the query
- **2** – Server failure - The name server was unable to process this query due to a problem with the name server
- **3** – Name Error - The domain name referenced in the query does not exist
- **4** – Not Implemented - The name server does not support the requested kind of query
- **5** – Refused - The name server refuses to perform the specified operation for policy reasons
- **6-15** – Reserved for future use

3 Application Design

3.1 Code Structure

The project is organized into clearly separated modules:

Listing 1: Project structure

```
1      baddomains.txt          # Example blocklist from
assignment
2      Makefile
3      README.md              # Project overview
4      manual.pdf             # This documentation
5      src
6          arguments.cpp       # Argument parsing
implementation
7          arguments.hpp       # Argument parsing interface
8          dns.cpp             # Main application entry point
9          dnsmessage.cpp      # DNS protocol implementation
10         dnsmessage.hpp      # DNS protocol interface
11         macro.hpp           # Debug macro
12         sock.cpp            # Socket implementation
13         sock.hpp            # Socket interface
14     tests
15         catch.hpp           # Catch2 testing framework
16         tests.cpp           # Unit tests
```

The debug macro used in this project was adapted from IPK Project 2 [4].

4 Implementation

4.1 Program Initialization

The program starts by creating instance of Arguments structure that parses arguments from command line and saves them. It also creates a vector of strings that contains the blocked domains. If domain name was provided in the arguments instead of IP address it resolves it. (`arguments.hpp`)

After this the program creates a Client instance with information from the arguments (port, resolved address and blocked domains). (`sock.hpp`)

In the constructor the Client creates 2 sockets for communicating with the user. One socket for IPv4 and the other one for IPv6. After they are created they are binded to the port that was specified in the arguments. Next the last socket is created, this one is used for communication with the upstream server. For this socket timeout is set so if the server doesn't respond the program doesn't freeze.

The program runs in a while loop and I use `select()` to monitor activity on both client sockets. Select also has timeout so it can check the while expression that will change when the Ctrl+C is pressed or program is terminated in other way. This is so

the program has graceful stop and frees the objects and closes the sockets as it should.
(`dns.cpp` main)

4.2 Query Handling

When client sends a query the select changes the file descriptor and client instance is called with function `handle_client` (`sock.hpp`). The function receives the query from the client using `recvfrom()` and checks if the query was sent from IPv4 or IPv6 interface and sets the structure `sockaddr_storage`. After this the `parse_dns_query()` is called.

This function parses the DNS query and saves the result into `DNSMessage` structure (`dnsmessage.hpp`). The function checks if the DNS query is malformed or that it must contain exactly 1 question and if it doesn't exception is thrown. On the forum it was said that more questions in 1 DNS query are generally not used.

Then the parsed DNS query name is compared with the blocked domains using `domain.blocked()` function. After this if the domain is blocked function `send_refused()` is called that creates a DNS Response header with status `REFUSED` and sends it to the client.

If the domain was not blocked the `forward_to_upstream()` function is called that forwards the DNS Query from the client to the upstream DNS server and waits for the response from the upstream server. If the server doesn't respond the timeout is triggered. Otherwise the response from the server is forwarded back to the client.

Since the buffer is big enough the program processes the queries sequentially like described here. I tested the program with 100 consecutive client queries and it worked fine.

5 User Manual

5.1 Compilation

To compile the program, run:

```
1 make
```

5.2 Usage

After compiling the program with `make` it can be run using:

```
1 ./dns -s <server> [-p <port>] -f <filter_file> [-v] [-h]
```

Parameters:

- `-s` – IP Address or hostname of upstream DNS resolver
- `-p` – Port for incoming queries (default: 53)
- `-f` – Path to filter file containing blocked domains

- `-h` – Help message
- `-v` – Verbose mode (prints debug information to stderr)

5.3 Example

```
1 ./dns -s dns.google.com -p 12345 -f blocked.txt
```

6 Testing

6.1 Test Environment

I developed the program on macOS but tested on eva and merlin as well. All tests passed on all platforms.

6.2 Unit Tests

The program was tested with Catch2[5] Unit Testing Framework. I decided to use Catch because I read about it in *C++ Crash Course* [6].

6.2.1 List of All Unit Tests

Listing 2: Unit test structure

```
1 TEST_CASE("CLI argument parsing", "[cli]")
2     SECTION("All arguments provided")
3     SECTION("Missing optional port argument")
4     SECTION("Missing required address argument")
5     SECTION("Invalid port number")
6     SECTION("Invalid port number out of range")
7     SECTION("Missing required file argument")
8     SECTION("Extra unknown argument")
9
10    TEST_CASE("Blocked domains", "[blocked_domains]")
11        SECTION("Domain is not blocked")
12        SECTION("Domain is blocked")
13        SECTION("Subdomain is blocked")
14
15    TEST_CASE("DNS message parsing", "[dns_message]")
16        SECTION("Valid DNS query")
17        SECTION("Query too short")
18        SECTION("Multiple questions rejected")
```

6.3 Manual Testing

I tested the program with unit tests as well as manual testing. When using the `-v` verbose flag the program prints all important information that was useful when testing the program manually. The program was tested on `eva` and `merlin` with unit tests as well as manually by starting the program and using `dig` to send the queries. The program passed all of the tests.

6.3.1 Testing Working Domain

```
1 xuhliar00@merlin: ~$ dig +short @localhost -p 15000 google.com
2 142.251.36.110
```

6.3.2 Testing Blocked Domain

```
1 xuhliar00@merlin: ~$ dig @localhost -p 15000 baddomain.org | grep
   status
2 ;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 22058
```

6.4 Debug Output

When running with `-v` verbose flag, the program provides detailed logging:

Listing 3: Debug information output

```
1 ./dns -s dns.google.com -p 15000 -f small_bad_domains.txt -v
2
3 src/dns.cpp:73 | main | Reading client IPv6 socket
4 src/sock.cpp:223 | handle_client | [IPv6] From [::1]:53023
   51 bytes
5 src/dnsmessage.cpp:53 | parse_dns_query | ID: 0xfc37, Flags: 0
   x0120
6 src/dnsmessage.cpp:56 | parse_dns_query | Question - QNAME:
   google.com
7 src/sock.cpp:241 | handle_client | Domain google.com is ALLOWED
8 src/sock.cpp:174 | forward_to_upstream | Forwarded query (51
   bytes)
9 src/sock.cpp:188 | forward_to_upstream | Received response (55
   bytes)
10 src/sock.cpp:196 | forward_to_upstream | Forwarded response (55
   bytes)
```

6.5 Memory Leak Testing

When running `valgrind` on `server merlin` and querying for domain after exiting the program `valgrind` shows no leaks:

Listing 4: Valgrind output

```
1 ==6604== HEAP SUMMARY:
2 ==6604==      in use at exit: 0 bytes in 0 blocks
3 ==6604==    total heap usage: 6,103 allocs, 6,103 frees,
4 ==6604==      581,928 bytes allocated
5 ==6604==
6 ==6604== All heap blocks were freed -- no leaks are possible
7 ==6604==
8 ==6604== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
   from 0)
```

7 Limitations

- The upstream DNS server must be reachable via IPv4 (IPv6 clients are supported).
- Blocked domain matching uses linear search – not optimal for very large lists.
- Only single-question DNS queries are accepted. (It was said on forum that this is okay)
- Program processes queries sequentially. I tested that it can still handle 100 consecutive clients even without implementing query response mapping.
- The resolver does not support TCP queries just UDP.

8 Conclusion

This project successfully implements a filtering DNS resolver that blocks blocked and forwards other queries to an upstream DNS server. The application supports both IPv4 and IPv6 clients, handles errors gracefully, and provides logging for debugging.

References

- [1] Wikipedia contributors, *Domain Name System*, Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Domain_Name_System
- [2] P. Mockapetris, *Domain names - implementation and specification*, RFC 1035, November 1987, <https://www.rfc-editor.org/rfc/rfc1035>
- [3] O'Reilly Media, *Hands-On Network Programming with C*, <https://www.oreilly.com/library/view/hands-on-network-programming/9781789349863/812dd5c5-0d22-4ccd-8faf-f339b416bb2e.xhtml>
- [4] *IPK Project 2: Client for a chat server using the IPK25-CHAT protocol*, Example of client logging in C, FIT VUT Brno, https://git.fit.vutbr.cz/NESFIT/IPK-Projects/src/branch/master/Project_2
- [5] Catch2, *A modern, C++-native, test framework for unit-tests*, <https://github.com/catchorg/Catch2>
- [6] Josh Lospinoso, *C++ Crash Course: A Fast-Paced Introduction*, No Starch Press, 2019